

Design Document —

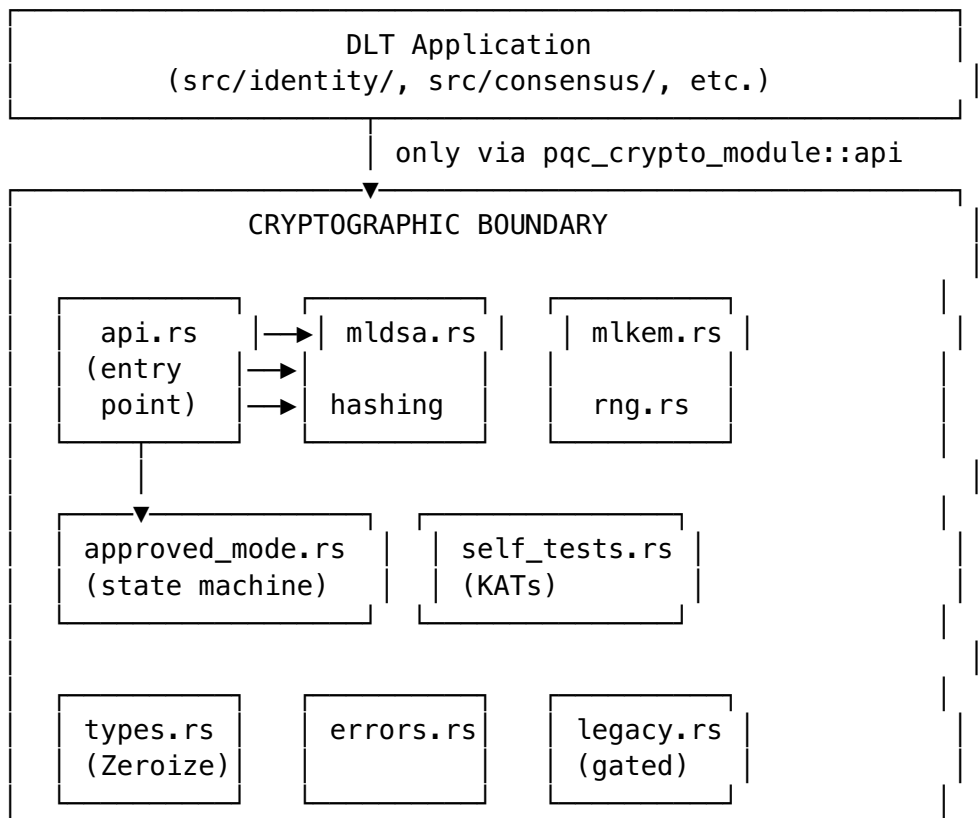
pqc_crypto_module v0.1.0

Disclaimer: This module is prepared for FIPS 140-3 evaluation and is not currently validated.

1. Purpose

pqc_crypto_module is a Rust crate that isolates all cryptographic operations for the Cerulean Ledger DLT behind a single, auditable boundary. It provides post-quantum digital signatures (ML-DSA-65), key encapsulation (ML-KEM-768), and hashing (SHA3-256) with approved-mode enforcement, startup self-tests, and key zeroization.

2. Architecture Overview



3. Cryptographic Boundary

The boundary is defined as all Rust source files under `crates/pqc_crypto_module/src/`. The Rust crate system enforces this: external code can only access pub items exported by the crate.

Files inside the boundary

File	Role
<code>lib.rs</code>	Crate root; re-exports public modules
<code>api.rs</code>	Public API entry point for all approved operations
<code>mldsa.rs</code>	ML-DSA-65 key generation, signing, verification
<code>mlkem.rs</code>	ML-KEM-768 key encapsulation (FIPS 203 via <code>pqcrypto-mlkem</code>)
<code>hashing.rs</code>	SHA3-256 hashing
<code>rng.rs</code>	CSPRNG wrapper with continuous test
<code>self_tests.rs</code>	Known Answer Tests
<code>approved_mode.rs</code>	State machine (AtomicU8) and enforcement guards
<code>types.rs</code>	Typed wrappers for keys, signatures, hashes with <code>ZeroizeOnDrop</code>
<code>errors.rs</code>	<code>CryptoError</code> enum via <code>thiserror</code>
<code>legacy.rs</code>	Non-approved algorithms (gated; outside approved boundary)

Files outside the boundary

Path	Role
<code>tests/*.rs</code>	Integration tests (verification only)
<code>Cargo.toml</code>	Build configuration
<code>*.md</code>	Documentation

Boundary enforcement

The boundary is enforced by:

1. **Rust module system:** Only pub items in `lib.rs` are accessible to external crates.
2. **pub(crate) visibility:** Internal functions (e.g., `*_raw` variants used by self-tests) are not accessible outside the crate.

3. **Integration tests:** `tests/api_boundary.rs` verifies that all operations fail before initialization. `tests/no_fallback.rs` verifies no classical algorithm fallback exists.

4. API Entry Points

All external access goes through `ppc_crypto_module::api`:

Function	Input	Output	Guard
<code>initialize_approved_mode()</code>	None	<code>Result<(), CryptoError></code>	Sets state to SelfTesting, runs KATs
<code>generate_mldsa_keypair()</code>	None	<code>Result<MldsaKeyPair, CryptoError></code>	requires approved
<code>sign_message(sk, msg)</code>	<code>&MldsaPrivateKey, &[u8]</code>	<code>Result<MldsaSignature, CryptoError></code>	requires approved
<code>verify_signature(pk, msg, sig)</code>	<code>&MldsaPublicKey, &[u8], &MldsaSignature</code>	<code>Result<(), CryptoError></code>	requires approved
<code>sha3_256(data)</code>	<code>&[u8]</code>	<code>Result<Hash256, CryptoError></code>	requires approved
<code>generate_mlkem_keypair()</code>	None	<code>Result<MlKemKeyPair, CryptoError></code>	requires approved
<code>mlkem_encapsulate(pk)</code>	<code>&MlKemPublicKey</code>	<code>Result<(MlKemCiphertext, MlKemSharedSecret), CryptoError></code>	requires approved
<code>mlkem_decapsulate(sk, ct)</code>	<code>&MlKemPrivateKey, &MlKemCiphertext</code>	<code>Result<MlKemSharedSecret, CryptoError></code>	requires approved
<code>random_bytes(n)</code>	<code>usize</code>	<code>Result<Vec<u8>, CryptoError></code>	requires approved

Every function except `initialize_approved_mode()` calls `require_approved()` as its first operation. If the module is not in Approved state, the function returns an error immediately without performing any cryptographic operation.

5. Data Flow

Initialization flow

```

Caller —> api::initialize_approved_mode()
          |
          |— set_state(SelfTesting)
          |— self_tests::run_all()
              |— kat_sha3_256()
              |— kat_mldsa65()
              |— kat_mlkem()
              |— test_rng()
  
```

```

      |
      | [all pass] → set_state(Approved) → Ok(())
      | [any fail] → set_state(Error)    →
Err(CryptoError)

```

Signing flow (representative of all approved operations)

```

Caller → api::sign_message(sk, msg)
      |
      | mlds::sign_message(sk, msg)
      |   |
      |   | require_approved() → [not Approved?]
      |   |
      |   | → Err
      |   |
      |   |   |
      |   |   | sign_message_raw(sk, msg)
      |   |   |   |
      |   |   |   | ppcrypto_mlds::mlds65::SecretKey::from_bytes(sk)
      |   |   |   |   |
      |   |   |   |   | ppcrypto_mlds::mlds65::detached_sign(msg, sk)
      |   |   |   |   |   |
      |   |   |   |   |   | MldsSignature(sig_bytes)
      |   |   |   |   |   |
      |   |   |   |   |   | Result<MldsSignature, CryptoError>

```

Legacy algorithm flow (blocked in Approved mode)

```

Caller → legacy::legacy_sha256(data)
      |
      | ensure_not_approved()
      |   |
      |   | [state == Approved] →
      |   |
      |   | Err(NonApprovedAlgorithm)
      |   |
      |   | [state != Approved] →
      |   |
      |   | sha2::Sha256::digest(data)

```

6. Internal Components

6.1 State Machine (approved_mode.rs)

A global AtomicU8 stores the module state. SeqCst ordering ensures visibility across threads. The state values are:

- 0 = Uninitialized
- 1 = SelfTesting
- 2 = Approved
- 3 = Error

Two guard functions enforce access:

- `require_approved()`: Returns `Ok(())` only if state is Approved.
- `ensure_not_approved()` (in `legacy.rs`): Returns `Ok(())` only if state is **not** Approved.

6.2 Self-Tests (`self_tests.rs`)

Runs four KATs sequentially. Any failure short-circuits and returns an error. See [SELF_TEST_DOCUMENTATION.md](#).

6.3 Type System (`types.rs`)

Wraps raw byte vectors in distinct newtypes (`MldsPublicKey`, `MldsPrivateKey`, `MldsSignature`, `Hash256`, etc.). Private keys and shared secrets derive `ZeroizeOnDrop` to overwrite memory on drop. Debug implementations for sensitive types print `[REDACTED]`.

6.4 Error Handling (`errors.rs`)

`CryptoError` is defined via `thiserror` with variants for each failure mode:

- `ModuleNotInitialized` — operation before `initialize_approved_mode()`
- `ModuleInErrorState` — operation after self-test failure
- `SelfTestFailed(String)` — specific KAT failure description
- `InvalidKey(String)` — key format or size error
- `InvalidSignature` — signature format error
- `VerificationFailed` — signature did not verify
- `RngFailure(String)` — CSPRNG error
- `NonApprovedAlgorithm` — legacy algorithm called in Approved mode
- `SerializationError(String)` — serialization failure

6.5 RNG (`rng.rs`)

Wraps `rand::rngs::OsRng` for all randomness. Provides:

- `fill_random(buf)` — fill a buffer with random bytes
- `random_bytes(n)` — allocate and fill `n` random bytes
- `continuous_rng_test()` — generate two 32-byte samples and verify they differ

6.6 Legacy Module (`legacy.rs`)

Contains non-approved algorithms (Ed25519, SHA-256, HMAC-SHA256) behind runtime guards. When compiled with `--features approved-only`, the entire module is excluded via `compile_error!`. See [NON_APPROVED_USAGE.md](#).

7. Dependencies

Crate	Version	Purpose
<code>pqcrypto-mldsa</code>	0.1.2	ML-DSA-65 implementation
<code>pqcrypto-traits</code>	0.3	Traits for PQC types
<code>sha3</code>	0.10	SHA3-256
<code>sha2</code>	0.10	SHA-256 (legacy only)

Crate	Version	Purpose
hmac	0.12	HMAC (legacy only)
ed25519-dalek	2.1	Ed25519 (legacy only)
rand	0.8	CSPRNG wrapper
rand_core	0.6	getrandom backend
zeroize	1.7	Memory zeroization
thiserror	1.0	Error type derivation
hex	0.4	Hex encoding for hash output

Dependencies sha2, hmac, and ed25519-dalek are used only by the legacy module and are excluded from the approved boundary.